

PRACTICAL WEEK-12

SESSION-12

TASK1: Implement the Producer-Consumer problem using counting semaphores and POSIX threads. Evaluate synchronization correctness and system throughput under different buffer sizes.

```
madhulika@LAPTOP-D2M8NKBH:~$ cd ~/OSSP/practical
madhulika@LAPTOP-D2M8NKBH:~/OSSP/practical$ mkdir -p week12/task1
madhulika@LAPTOP-D2M8NKBH:~/OSSP/practical$ cd week12/task1
madhulika@LAPTOP-D2M8NKBH:~/OSSP/practical/week12/task1$ nano producer_consumer.c
madhulika@LAPTOP-D2M8NKBH:~/OSSP/practical/week12/task1$ gcc producer_consumer.c -o T1 -pthread
madhulika@LAPTOP-D2M8NKBH:~/OSSP/practical/week12/task1$ ls -l
total 28
-rwxr-xr-x 1 madhulika madhulika 17096 Sep 19 03:24 T1
-rw-r--r-- 1 madhulika madhulika 5086 Sep 19 03:24 producer_consumer.c
madhulika@LAPTOP-D2M8NKBH:~/OSSP/practical/week12/task1$ ./T1
=== Practical Week 12 - Task 1 ===
Producer-Consumer Using Counting Semaphores

Producers : 2
Consumers : 2
Total items: 10000

-----
Buffer Size      Time (seconds)  Throughput
-----
Consumer 1 consumed 5000 items.
Consumer 2 consumed 5000 items.
5                0.094005      106377.75 items/sec
Consumer 1 consumed 5000 items.
Consumer 2 consumed 5000 items.
10              0.048493      206599.27 items/sec
Consumer 2 consumed 5000 items.
Consumer 1 consumed 5000 items.
50              0.010861      920699.51 items/sec
Consumer 2 consumed 5000 items.
Consumer 1 consumed 5000 items.
100             0.011736      852073.34 items/sec
-----

Synchronization test completed successfully.
All produced items were consumed.
No buffer overflow or underflow occurred.
madhulika@LAPTOP-D2M8NKBH:~/OSSP/practical/week12/task1$
```

```
GNU nano 8.7.1 producer_consumer.c *
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>
#include <time.h>

#define NUM_ITEMS 10000
#define NUM_PRODUCERS 2
#define NUM_CONSUMERS 2

int *buffer;
int buffer_size;

int in = 0;
int out = 0;

sem_t empty_slots;
sem_t full_slots;

pthread_mutex_t mutex;

void *producer(void *arg)
{
    int producer_id = *(int *)arg;

    for (int i = 0; i < NUM_ITEMS / NUM_PRODUCERS; i++)
    {
        int item = producer_id * 100000 + i;

        /*
         * Wait until an empty slot is available.
         */
        sem_wait(&empty_slots);

        /*
         * Protect buffer access.
         */
    }
}
```

```
GNU nano 8.7.1 producer_consumer.c *
{
    int sizes[] = {5, 10, 50, 100};
    int num_sizes = sizeof(sizes) / sizeof(sizes[0]);

    printf("=== Practical Week 12 - Task 1 ===\n");
    printf("Producer-Consumer Using Counting Semaphores\n\n");

    printf("Producers : %d\n", NUM_PRODUCERS);
    printf("Consumers : %d\n", NUM_CONSUMERS);
    printf("Total items: %d\n\n", NUM_ITEMS);

    printf("-----\n");
    printf("Buffer Size\tTime (seconds)\tThroughput\n");
    printf("-----\n");

    for (int i = 0; i < num_sizes; i++)
    {
        double time_taken = run_test(sizes[i]);

        double throughput =
            NUM_ITEMS / time_taken;

        printf("%d\t\t%.6f\t\t%.2f items/sec\n",
            sizes[i],
            time_taken,
            throughput);
    }

    printf("-----\n");

    printf("\nSynchronization test completed successfully.\n");
    printf("All produced items were consumed.\n");
    printf("No buffer overflow or underflow occurred.\n");

    return 0;
}

G Help      ^O Write Out ^F Where Is  ^K Cut       ^T Execute
X Exit      ^R Read File ^\ Replace  ^U Paste    ^J Justify
```

Observation: The Producer-Consumer problem was successfully implemented using POSIX threads and counting semaphores. Multiple producer threads generated items and placed them into a shared bounded buffer, while consumer threads removed the items from the buffer. The empty_slots and full_slots counting semaphores controlled the availability of buffer positions and produced items, while a mutex protected simultaneous access to the shared buffer. The program was tested with different buffer sizes, and the execution time and throughput were measured for each configuration. All produced items were successfully consumed without buffer overflow or underflow, demonstrating correct synchronization.

Task2: Design a deadlock scenario involving multiple threads competing for shared resources. Demonstrate deadlock occurrence, identify the four necessary conditions for deadlock, and implement a deadlock prevention strategy using resource ordering.

```
madhulika@LAPTOP-D2M8NKBH:~/practical/week12/task2$ nano deadlock_prevention
.c
madhulika@LAPTOP-D2M8NKBH:~/practical/week12/task2$ gcc deadlock_prevention.
c -o T2 -pthread
madhulika@LAPTOP-D2M8NKBH:~/practical/week12/task2$ ls -l
total 28
-rwxr-xr-x 1 madhulika madhulika 16632 Sep 19 03:42 T2
-rw-r--r-- 1 madhulika madhulika 5558 Sep 19 03:41 deadlock_prevention.c
madhulika@LAPTOP-D2M8NKBH:~/practical/week12/task2$ ./T2 deadlock
=== Practical Week 12 - Task 2 ===
Deadlock and Deadlock Prevention

=====
DEADLOCK DEMONSTRATION
=====

Two threads will request resources in opposite order.
The threads will become blocked waiting for each other.

Thread 1: requesting Resource 1...
Thread 1: acquired Resource 1.
Thread 2: requesting Resource 2...
Thread 2: acquired Resource 2.
Thread 2: requesting Resource 1...
Thread 2: waiting for Resource 1...
Thread 1: requesting Resource 2...
Thread 1: waiting for Resource 2...

-----
DEADLOCK DETECTED
-----

Thread 1 holds Resource 1 and waits for Resource 2.
Thread 2 holds Resource 2 and waits for Resource 1.
Neither thread can continue.

The program will terminate this demonstration.
This avoids leaving a permanently blocked process.
```

```

madhuLiKa@LAPTOP-D2M8NKBH:~/practical/week12/task2$ ./T2 ordered
=== Practical Week 12 - Task 2 ===
Deadlock and Deadlock Prevention

=====
DEADLOCK PREVENTION
RESOURCE ORDERING
=====

Resource ordering rule:
Every thread must acquire Resource 1 before Resource 2.

Thread 1: requesting Resource 1...
Thread 1: acquired Resource 1.
Thread 2: requesting Resource 1...
Thread 1: requesting Resource 2...
Thread 1: acquired Resource 2.
Thread 1: using both resources.
Thread 1: released both resources.
Thread 2: acquired Resource 1.
Thread 2: requesting Resource 2...
Thread 2: acquired Resource 2.
Thread 2: using both resources.
Thread 2: released both resources.

-----
DEADLOCK PREVENTED
-----

Both threads followed the same resource order.
No circular waiting occurred.
All resources were released successfully.

```

```

GNU nano 8.7.1      deadlock_prevention.c *
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <unistd.h>
#include <time.h>

pthread_mutex_t resource1 = PTHREAD_MUTEX_INITIALIZER;
pthread_mutex_t resource2 = PTHREAD_MUTEX_INITIALIZER;

/*
 * -----
 * DEADLOCK DEMONSTRATION
 * -----
 */

void *deadlock_thread1(void *arg)
{
    printf("Thread 1: requesting Resource 1...\n");
    pthread_mutex_lock(&resource1);

    printf("Thread 1: acquired Resource 1.\n");

    sleep(1);

    printf("Thread 1: requesting Resource 2...\n");
    printf("Thread 1: waiting for Resource 2...\n");

    pthread_mutex_lock(&resource2);

    printf("Thread 1: acquired Resource 2.\n");

    pthread_mutex_unlock(&resource2);
    pthread_mutex_unlock(&resource1);

    return NULL;
}

```

```

GNU nano 8.7.1          deadlock_prevention.c *
| printf("Thread 2: acquired Resource 2.\n");
|
| printf("Thread 2: using both resources.\n");
|
| pthread_mutex_unlock(&resource2);
| pthread_mutex_unlock(&resource1);
|
| printf("Thread 2: released both resources.\n");
|
| return NULL;
|
}

void demonstrate_ordering()
{
    pthread_t thread1;
    pthread_t thread2;

    printf("\n=====\n");
    printf("DEADLOCK PREVENTION\n");
    printf("RESOURCE ORDERING\n");
    printf("=====\n\n");

    printf("Resource ordering rule:\n");
    printf("Every thread must acquire Resource 1 before Resource 2.\n\n");

    pthread_create(&thread1, NULL, ordered_thread1, NULL);
    pthread_create(&thread2, NULL, ordered_thread2, NULL);

    pthread_join(thread1, NULL);
    pthread_join(thread2, NULL);

    printf("\n-----\n");
    printf("DEADLOCK PREVENTED\n");
    printf("-----\n");

    printf("Both threads followed the same resource order.\n");
    printf("No circular waiting occurred.\n");
}

^G Help      ^O Write Out  ^F Where Is   ^K Cut        ^T Execute
^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify

```

```

GNU nano 8.7.1          deadlock_prevention.c *
/*
 * -----
 * MAIN
 * -----
 */

int main(int argc, char *argv[])
{
    printf("=== Practical Week 12 - Task 2 ===\n");
    printf("Deadlock and Deadlock Prevention\n");

    if (argc != 2)
    {
        printf("\nUsage:\n");
        printf("  ./T2 deadlock\n");
        printf("  ./T2 ordered\n");
        return 1;
    }

    if (strcmp(argv[1], "deadlock") == 0)
    {
        demonstrate_deadlock();
    }
    else if (strcmp(argv[1], "ordered") == 0)
    {
        demonstrate_ordering();
    }
    else
    {
        printf("\nInvalid option.\n");
        printf("Use 'deadlock' or 'ordered'.\n");
        return 1;
    }

    return 0;
}

^G Help      ^O Write Out  ^F Where Is   ^K Cut        ^T Execute
^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify

```

```
GNU nano 8.7.1      deadlock_prevention.c *
/*
 * -----
 * MAIN
 * -----
 */

int main(int argc, char *argv[])
{
    printf("=== Practical Week 12 - Task 2 ===\n");
    printf("Deadlock and Deadlock Prevention\n");

    if (argc != 2)
    {
        printf("\nUsage:\n");
        printf("  ./T2 deadlock\n");
        printf("  ./T2 ordered\n");
        return 1;
    }

    if (strcmp(argv[1], "deadlock") == 0)
    {
        demonstrate_deadlock();
    }
    else if (strcmp(argv[1], "ordered") == 0)
    {
        demonstrate_ordering();
    }
    else
    {
        printf("\nInvalid option.\n");
        printf("Use 'deadlock' or 'ordered'.\n");
        return 1;
    }

    return 0;
}

^G Help      ^O Write Out  ^F Where Is   ^K Cut        ^T Execute
^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify
```

Observation: The program successfully demonstrated a deadlock situation involving two POSIX threads competing for two shared resources. In the deadlock scenario, one thread acquired Resource 1 and waited for Resource 2, while the other thread acquired Resource 2 and waited for Resource 1, resulting in circular waiting. The four necessary conditions for deadlock—mutual exclusion, hold and wait, no preemption, and circular wait—were present. Deadlock prevention was then implemented using resource ordering, where both threads acquired Resource 1 before Resource 2. This prevented circular waiting and allowed both threads to complete successfully.